

SHA-256 GPU vs CPU — Benchmark Results

Owner: Mudrik · **Task:** scale up, benchmark, and chart GPU vs CPU throughput (`results/`).

This document records the throughput benchmark for the parallel SHA-256 engine (one GPU thread per message, per `IO_CONTRACT.md`). Every number here was produced on real hardware and is reproducible from the harness and CSVs in this folder — nothing is hand-typed.

Headline: the GPU hashes 1,000,000 real messages **~18× faster end-to-end** and **~63× faster kernel-only** than a serial CPU baseline, and the GPU output matches the project’s `expected_digests.bin` **byte-for-byte**.

1. Environment

Item	Value
GPU	NVIDIA GeForce RTX 4060 Laptop GPU (24 SMs, 8.6 GB, <code>sm_89</code>)
CUDA	runtime 13.2, driver 13.2 (toolkit <code>nvcc</code> 13.0)
Host compiler	MSVC 14.44 (VS 2022 Build Tools)
OS	Windows 11
CPU baseline	single-threaded scalar SHA-256 (FIPS 180-4), one message at a time
Dataset	<code>data/</code> — 1,000,000 messages, ~32-byte avg (the real Karan-generated set) + synthetic sweeps

The CPU baseline is a self-contained reference (no OpenSSL dependency). It is **verified against the three NIST vectors** from the I/O contract at startup — the harness aborts before timing anything if they don’t match. This run: all three OK.

2. Methodology (what makes these numbers robust)

The benchmark is **not** a single timed run. For each data point:

- **Warm-up is sustained, not one-shot.** Before timing, the kernel is hammered for ~300 ms to force the GPU to its boost clocks. A laptop GPU idle-downclocks between segments, so a single warm-up iteration measures *cold* clocks — this was visibly contaminating sub-millisecond kernels until fixed (a 100k-message point briefly read slower than both 10k and 1M).
- **Many repeats, reported as statistics.** Each point is repeated 10–50× (more for small N). We report **min / median / mean / stddev**; the **median** is the headline figure since it’s robust to one-off OS/GPU hiccups.
- **GPU time is split by stage** using CUDA events: Host→Device copy, kernel, Device→Host copy. This separates *compute* throughput from *transfer* cost.

- **Correctness gate on every point.** GPU digests are compared to the CPU reference for all messages; for the real `data/` set they are *also* compared to `expected_digests.bin`. A mismatch is reported, never hidden.
- **Three measurement layers** (smallest → largest scope):
- **kernel-only** = pure compute (what the parallelism actually buys you);
- **device end-to-end** = H2D + kernel + D2H with device buffers **allocated once and reused** (steady-state transfer + compute);
- **production full-call** = one `sha256_gpu_hash()` invocation = `cudaMalloc` → H2D → kernel → D2H → `cudaFree`, i.e. what `validate`, `hash_dataset`, and `make benchmark` actually pay **per call**. The first two exclude one-time allocation (fair for *repeated* hashing into the same buffers); the third includes it (fair for *one-shot* API calls). §4a reports the production full-call path explicitly so neither view is misleading.

The harness emits machine-readable CSVs:

File	Contents
<code>benchmark_summary.csv</code>	scaling log — one row per dataset size (median times, hashes/sec, GB/s, speedup)
<code>benchmark_<N>.csv</code>	per-run snapshot — <i>every</i> timed repeat at size N (raw H2D/kernel/D2H/total ms), the distribution the medians are drawn from
<code>benchmark_realdata.csv</code>	the real <code>data/</code> set, with the <code>expected_digests.bin</code> correctness verdict
<code>benchmark_blocksize.csv</code>	one row per block size at N=1,000,000 (min/med/sd)

These are regenerated on each run (gitignored per `results/*.csv`), so this document plus the captured console log `benchmark_run.txt` are the committed record.

3. Correctness (must pass before any speed claim)

Check	Result
Host SHA-256 vs NIST vectors (<code>"", "abc"</code> , 56-byte)	✔ all match
GPU vs CPU reference (1M real + every synthetic size)	✔ 0 mismatches
GPU vs project <code>data/expected_digests.bin</code> (1M)	✔ byte-for-byte match

Per the project rule — *correctness before speed* — the speed numbers below are only meaningful because this section passes.

4. Real dataset (`data/` , 1,000,000 messages)

Device end-to-end GPU time broken into stages (median, device buffers reused — *excludes* per-call `cudaMalloc` / `cudaFree` ; see §4a for the full production call):

Stage	Time (ms)	Share
Host→Device copy	3.59	31%
Kernel (compute)	3.39	29%
Device→Host copy	4.54	39%
Device end-to-end total	11.55	100%

Metric	CPU	GPU	Speedup
Time (median)	212.2 ms	11.55 ms (e2e) / 3.39 ms (kernel)	18.4× / 62.5×
Throughput	4.7 M hashes/s	86.6 M hashes/s (e2e)	18.4×
Kernel bandwidth	—	9.43 GB/s	—

Key insight: for this message size the kernel is only ~29% of wall time — **PCIe transfer dominates** (~70%). The compute is ~63× faster than the CPU; the practical end-to-end win (~18×) is gated by getting data on and off the device.

4a. Production full-call path (`sha256_gpu_hash` , incl. `cudaMalloc` / `cudaFree`)

The §4 numbers reuse device buffers, so they measure *repeated* hashing. The production API in `src/kernel/sha256_gpu.cu` — the function `validate` , `hash_dataset` , and `make benchmark` all call — does `cudaMalloc` → H2D → kernel → D2H → `cudaFree` on **every** invocation. Measuring that actual function (1M data/ messages, median of 20 calls, same GPU) gives the honest one-shot cost:

Path	Time	Throughput	GB/s	Speedup vs CPU
<code>sha256_gpu_hash()</code> full call (malloc→...→free, median of 20)	25.19 ms	39.7 M/s	1.27	8.5× ^a
device end-to-end (buffers reused, §4)	17.42 ms ¹	57.4 M/s	1.84	12.3×
CPU scalar reference (1 thread)	214.9 ms	4.7 M/s	0.15	—

Per-call `cudaMalloc` + `cudaFree` overhead = 7.77 ms ≈ 31 % of the full call. Output MATCHED both the CPU reference and `data/expected_digests.bin` . So the defensible headline for the *production API as written* is **~8.5× end-to-end** (not 18×); the larger figures are real but describe compute (~63×) or amortized, buffer-reuse hashing (~18×), not a single `sha256_gpu_hash()` call.

^a CPU baseline here is a scalar loop, not OpenSSL — see the `make benchmark` row below for the OpenSSL comparison.

¹ This 17.42 ms is wall-clock around the full reused-buffer iteration (incl. allocating the host output vector each call), so it runs a touch higher than the 11.55 ms CUDA-event sum in §4, which times only the three device operations. Same workload, two valid stopwatches — the gap is host-side, not GPU.

The fix is not kernel tuning — it's allocation. Reusing device buffers across calls (or a pinned-memory pool) removes the 31 % `cudaMalloc / cudaFree` tax and recovers the §4 end-to-end rate; that, not occupancy, is the next lever.

`make benchmark` (Mohshinsha's `benchmark.cpp`, OpenSSL baseline)

The team's `make benchmark` harness calls the **same** `sha256_gpu_hash()` and uses **OpenSSL `SHA256()`** for the CPU baseline. This box has no `make /OpenSSL` by default, so it was reproduced by compiling `benchmark.cpp + sha256_gpu.cu` directly with `nvcc` against an installed Win64 OpenSSL (build/run lines in `benchmark_makebench_run.txt`, CSV `benchmark_makebench_1000000.csv`):

Mode (1M <code>data/</code>)	Time	Throughput	Speedup
CPU (OpenSSL <code>SHA256()</code> , serial)	314.93 ms	3.18 M/s	—
GPU <code>sha256_gpu_hash()</code> (full prod. call)	19.98 ms	50.0 M/s	15.76×

`benchmark.cpp` times a single call after one warm-up; its 19.98 ms lands inside the production full-call band above (the 20-repeat median is 25.19 ms). Notably the **OpenSSL one-shot CPU baseline is slower (315 ms) than the tight scalar loop (215 ms)** — for 1M tiny (~32-byte) messages, OpenSSL's per-call context init/teardown dominates, so its higher speedup (15.8×) reflects a slower CPU baseline, not a faster GPU. The honest production-path GPU number is the constant across both harnesses: **~20–25 ms / ~8–16× end-to-end per call**, gated by allocation + transfer.

5. Scaling sweep (synthetic, ~32-byte messages)

Median times; headline metrics in `benchmark_summary.csv`, raw per-repeat timings in the per-N snapshots (e.g. `benchmark_1000000.csv`).

Messages	Input	CPU (ms)	GPU e2e (ms)	GPU kernel (ms)	Speedup e2e	Speedup kernel	Correct
10,000	0.32 MB	2.16	0.190	0.041	11.4×	52.8×	✓
100,000	3.25 MB	20.13	1.239	0.371	16.3×	54.3×	✓
1,000,000	32.5 MB	202.30	11.78	3.438	17.2×	58.8×	✓
10,000,000	325 MB	2035.50	119.84	34.09	17.0×	59.7×	✓

Observations: - **CPU scales linearly** at a flat ~4.9 M hashes/s regardless of N — a serial loop has no economies of scale. - **GPU needs work to fill the device.** At 10k messages the launch + transfer overhead caps the end-to-end win at 11.4×; by 100k–1M the GPU is saturated and kernel-only speedup settles around **55–60×**. - **End-to-end speedup plateaus at ~16–17×** once transfer-bound, while kernel-only stays ~55–60× — re-confirming that PCIe, not compute, is the ceiling.

Throughput curve (kernel-only speedup vs CPU)

CPU	~4.9M	#		(flat, all sizes)
GPU	10k	#####	~53×	
GPU	100k	#####	~54×	
GPU	1M	#####	~59×	← saturated
GPU	10M	#####	~60×	

6. Block-size sweep (N = 1,000,000, kernel-only)

Threads/block	Kernel (ms, med)	Hashes/s	GB/s
64	3.28	305 M	9.91
128	3.39	295 M	9.60
256	3.47	288 M	9.36
512	3.53	283 M	9.20
1024	3.49	287 M	9.31

The kernel is **largely insensitive to block size** — all configs land within a tight ~7% band (3.28–3.53 ms). Smaller blocks edge ahead here (64 is fastest, and its stddev is the lowest in the CSV), but the gap is small enough to be a weak preference, not a tuning lever: the workload is bound by global-memory traffic and PCIe, not occupancy. The default `threadsPerBlock=256` from the I/O contract is a reasonable, safe choice. Data: [benchmark_blocksize.csv](#).

7. Conclusions

- 1. **Correctness holds at scale** — 1M GPU digests match the CPU reference *and* the project’s `expected_digests.bin` exactly; every synthetic size matches too.
- 2. **Compute is ~55–63× faster** than a serial CPU once the device is saturated (≥100k messages).
- 3. **End-to-end is ~16–18×** because host↔device transfer is ~70% of wall time for these short messages — the kernel finishes faster than the data can move.
- 4. **Block size barely matters**; `256` is fine.
- 5. **To go faster** the next lever is *transfer*, not compute: overlap copies with compute via CUDA streams / pinned memory, or keep data resident on the GPU across stages — not kernel micro-optimization.

8. Reproduce

```
# Linux / Colab:
nvcc -O3 -std=c++17 -Iinclude results/scale_benchmark.cu -o build/scale_benchmark
./build/scale_benchmark data          # data dir optional (default: data)

# Windows (from a VS Build Tools / "x64 Native Tools" shell):
nvcc -O3 -std=c++17 -Iinclude results\scale_benchmark.cu -o build\scale_benchmark.exe
.\build\scale_benchmark.exe data
```

Harness: [scale_benchmark.cu](#). It writes the CSVs above (`benchmark_summary.csv` , a `benchmark_<N>.csv` per size, plus the real-data and block-size files) and prints the console summary. Numbers in this document are from one canonical run on the machine in §1; expect small run-to-run variance on a laptop GPU ($\pm 10\%$ on kernel times due to clock/thermal behavior) — the medians and the conclusions are stable across runs.